# Confidence and Calibration of Activation Oracles for Reliable Interpretation of Language Model Internals

**Federico Torrielli**
University of Turin
federico.torrielli@unito.it

**Peter Schneider-Kamp**
University of Southern Denmark
petersk@imada.sdu.dk

**Lukas Galke Poech**
University of Southern Denmark
galke@imada.sdu.dk

arXiv:2605.26045v2 [cs.CL] 3 Aug 2026

## Abstract

An activation oracle is a language model trained to read another model's internal activations and describe them in natural language, for example to name a secret word the other model was trained to hide. Oracle answers carry no measure of confidence, which limits their use in auditing. We compare five ways of attaching a confidence score to an oracle's answer on this secret-word task, across four oracles from two model families (Qwen and Gemma, 8B to 27B parameters), at 6,000 samples per method and oracle. The five methods rank the same way on all four oracles. Which method to use depends on one question: can the auditor list the possible answers in advance? If the auditor can, then having the oracle score each candidate answer roughly doubles accuracy and separates correct from wrong answers best of the five (AUROC 0.92 to 0.96). If the oracle must generate its answer freely and no labeled data exists, the agreement rate over twenty samples is the only confidence that is calibrated on every oracle. Once labeled data exists, a rescaled answer probability reaches the same calibration at one generation instead of twenty. Asking the oracle to state a confidence number gives no usable signal on any oracle. Code and the patched trainer are available at https://github.com/federicotorrielli/probabilistic_activation_oracles.

## 1 Introduction

A large language model holds information in its internal activity while it works, and some of that information never shows up in the text it writes. An activation oracle is a second model trained to look at that internal activity and put it into words (Karvonen et al., 2025) (fig. 1, top). To test it, Karvonen et al. (2025) play a guessing game. They take a model that has been fine-tuned to keep a secret word, let the oracle look at that model's internal activity, and ask the oracle to name the word. Sometimes the oracle is right, sometimes it is wrong. This paper asks a different question: does the oracle know when it is right? A confidence score is called well calibrated when it matches how often the answer is in fact correct. We compare five ways of producing such a score and report which ones are calibrated.

The question matters because of what oracles are being proposed for: alignment auditing (Bricken et al., 2025; Sheshadri et al., 2026), deception detection (Ravindran, 2025), and elicitation of hidden objectives (Dietz et al., 2026). Each of these ends in a yes-or-no decision made by comparing a confidence score against a threshold: an auditor flags or releases a model, a monitoring pipeline routes or drops a generation. A miscalibrated score places that threshold wrong, so the pipeline acts on confident answers that are wrong or discards usable ones. Karvonen et al. (2025) name the lack of calibration as a main limitation of their oracles, and Jakkli et al. (2026) list missing calibration training among the reasons current oracles are hard to use on safety tasks. So far, no one has measured which confidence methods actually work on an oracle.

There are standard ways to attach a confidence score to a language model's output, but they were developed on unmodified models. An oracle is a steered model: while it answers, vectors taken from another model are added into part of its internal state. Whether the standard methods still work under that intervention is an empirical question. We answer it on the 20-word secret-word task of Karvonen et al. (2025), on four oracles spanning two model families and a 3.3× size range: Qwen3-8B and Gemma-2-9B (released by Karvonen et al., 2025), Gemma-3-27B, and a Qwen3.6-27B oracle we train and release. The five methods adapt established baselines: the probability the oracle assigned to its answer, agreement across twenty sampled answers (a short-answer form of self-consistency, Wang et al., 2022), asking the oracle for a confi-




Figure 1: An activation oracle names a hidden word and attaches no confidence to it. We compare five ways of supplying one. †: readouts introduced or first evaluated in this work. Three further readouts built for the steered setting are reported in section J.

dence number, in free text or over a fixed set of confidence labels (Kadavath et al., 2022; Lin et al., 2022), and scoring the 20 candidate words directly. We also designed three readouts that use the steering machinery itself, and all three carry weaker confidence than the five on every oracle; section J reports them with the mechanism behind each result.

The five methods rank the same way on all four oracles, so none of the advice below depends on which oracle an auditor is reading. The auditor's own situation does vary, and it splits the advice on one question: can the auditor list the possible answers in advance? When the 20 candidate words are known, letting the oracle score each of them directly is better on every measure: accuracy roughly doubles relative to generating an answer (e.g. 0.65 vs. 0.41 on Qwen3-8B, 0.53 vs. 0.23 on Qwen3.6-27B), and the resulting confidence separates correct from wrong answers best (AUROC 0.92 to 0.96). This confirms, with calibration numbers, the conjecture of Jakkli et al. (2026) that an oracle gives up more of what it knows when its output format is constrained than when it generates freely. When the answer must be generated freely and no labeled data is available, the agreement rate over twenty sampled answers (the bootstrap, at a tuned sampling temperature) is the only confidence that is calibrated on every oracle (expected calibration error, ECE, 0.04 to 0.12); every competing method is well calibrated on some oracles and far off on others. Once labeled data exists, a correction fitted to the answer's probability reaches the same calibration with a single generation instead of twenty. Asking the oracle for a confidence number gives no usable signal on any oracle (AUROC 0.40 to 0.53), and on Qwen3.6-27B the stated number is on average higher when the answer is wrong. This replicates, on steered oracles, a known gap on ordinary models: the uncertainty signal is present in the activations but does not reach the model's spoken confidence (Yuan et al., 2026; Miao and Ungar, 2026). Consistent with that reading, a constrained version of the same question, which scores five fixed confidence labels with the oracle's own output probabilities, recovers a usable signal (best-readout AUROC 0.68 to 0.82).

**Contributions.**

- The first study on uncertainty quantification for activation oracles: five confidence estimation methods compared head to head, plus three further readouts built for the steered setting in section J, on four oracles from two model families, against five metrics, at 6,000 samples per method-oracle pair, with bootstrap confidence intervals and a controlled target-set-size variant. Which methods work, and how well, is stable across family and scale.
- A testbed for this question, released with the paper. It holds the four oracles, twenty secret-word target models for each, the 6,000 sample

protocol, and a scorer that takes a list of answer and confidence pairs and returns accuracy, expected calibration error, Brier score, negative log-likelihood, and AUROC with bootstrap intervals. A new confidence method joins the comparison by emitting those pairs, and the numbers reported here are what it is measured against. We also release the first activation oracle for a hybrid attention architecture, Qwen3.6-27B, together with the trainer changes it required and the forty secret-word target models we retrained.
- A practical decision rule with three cases. If the candidate answers can be listed, have the oracle score them directly. If the answer must be generated and no labeled data exists, use the agreement rate over sampled answers, with the sampling temperature chosen so that mean agreement matches accuracy on a held-out slice. Once labeled data exists, a rescaled answer probability gives the same calibration at $1/20$th of the generation cost.
- A replication, on steered oracles, of the finding that activations carry an uncertainty signal the model's spoken confidence does not (Yuan et al., 2026; Miao and Ungar, 2026), together with a constrained-label readout, evaluated at full scale on all four oracles, that recovers the signal the free-text channel loses.

## 2 Background and Related Work

**Describing activations in words.** A transformer stores information about its input in the hidden states of its residual stream. Several methods take such a hidden state and return a description of it in natural language. The logit lens (nostalgebraist, 2020) applies the model's own output layer to a hidden state taken at an intermediate layer, and returns the resulting distribution over vocabulary tokens. The logit lens needs no training, and its description is a ranked list of tokens, so the logit lens reports only which token the model is about to predict. The activation difference lens (Minder et al., 2026) subtracts the activations of a base model from the activations of a model fine-tuned from it. Minder et al. (2026) show that a description of this difference names the topic the fine-tuning taught, and that on a model fine-tuned to hide a word it names the hidden word. The activation difference lens describes a model. The methods in this paper describe one hidden state at a time.

An activation oracle is a language model trained to accept another model's hidden states as input and answer questions about them in natural language. Pan et al. (2024) introduce this design: they build pairs of an activation vector and a written answer about that vector, then fine-tune a decoder model on those pairs. Karvonen et al. (2025) train a decoder the same way on a wider mixture of tasks, call the result an activation oracle, and test it on inputs far from its training data. The four oracles we benchmark are of this kind. Section 3.1 states how a hidden state reaches an oracle and how the oracle answers.

**Why oracle answers need a confidence.** The oracle's output is a plain-text sentence with no notion of certainty attached. Karvonen et al. (2025) observe that their model "will frequently produce an answer even when confidence is low". Jakkli et al. (2026) test the released Qwen3 oracles on safety-relevant tasks, find that roughly half of free-form answers are too generic to falsify, and identify the absence of calibration training as one of three fixable causes. They also conjecture that constrained output formats (multiple choice, binary, scored logits) extract more signal from an oracle than free-form generation. Our forced-choice and constrained-label results test that conjecture directly. In parallel, Bauer et al. (2026) improve the oracle training recipe and, in their evaluation suite, mitigate hallucination at inference time by sampling $k$ answers and keeping only those above an agreement threshold. That filter is the same signal as our bootstrap method; we contribute the calibration study of it, that is, whether the agreement rate can be read as a probability. An oracle reads a target model's hidden state by adding a rescaled copy of that hidden state to its own residual stream. Activation steering methods perform the same addition on a model's residual stream to change how the model behaves, for example to make it more cautious (Turner et al., 2024; Rimsky et al., 2024; Zou et al., 2025). The purpose is what differs: a steering method picks the added vector to obtain a behavior, an oracle adds the hidden state in order to describe it, and the target model is left untouched.

**Uncertainty for language models.** Modern neural networks are systematically miscalibrated, and a simple post-hoc rescale on labeled data reduces the error (Guo et al., 2017). Post-hoc temperature scaling is a one-knob correction: on a small set of examples with known answers, find the single number that makes the reported confidences match

reality best, and divide by it. We report both native calibration (no rescale) and post-hoc calibration (section 5.4). Kadavath et al. (2022) show that large models assign well-calibrated probabilities on multiple-choice questions and that a model's probability of the token "True", when asked to evaluate its own proposed answer, is an informative score. Lin et al. (2022) train models to verbalize calibrated confidence, but the calibration does not transfer to held-out task families without further training; without such training, verbalized confidence is systematically overconfident (Mielke et al., 2022; Xiong et al., 2024). Wang et al. (2022) introduce self-consistency: sample $k$ reasoning traces, take the most frequent final answer, and read the agreement rate as a confidence. Our bootstrap method is the short-answer form of this idea.

**Probe versus verbalized confidence.** Yuan et al. (2026) and Miao and Ungar (2026) report that the uncertainty signal present in a model's activations is largely unreachable through the model's spoken confidence: a linear probe on hidden states predicts correctness at AUROC up to 0.95 while the verbalized confidence is not informative about correctness, and the verbalized-confidence direction is roughly orthogonal to the calibration direction in activation space. We observe the same gap on steered oracles (section 5.5) and show that a constrained label readout, which scores a fixed set of answers with the model's own logits instead of letting it generate, recovers part of the signal. Basu et al. (2026) report a parallel gap on a clinical task.

# 3 Methods

## 3.1 Preliminaries

We distinguish between the *target model*, which is the model under inspection, and the *oracle*, which is the model that reads the target's hidden states and answers questions about them.

The target model first processes a prompt. We record the target's residual stream at one designated layer, at the last $K{=}10$ token positions of that prompt, denoted $h_1, \ldots, h_K \in \mathbb{R}^d$. The oracle receives a separate, and potentially different prompt. That prompt opens with the line `Layer:` $\ell$, continues with $K$ placeholder tokens, and ends with the question the oracle is instructed to answer. The oracle then runs its own prompt up to its injection layer, which gives the residual stream vectors $r_1, \ldots, r_K \in \mathbb{R}^d$ that the oracle holds at the $K$ placeholder positions. Before the oracle generates anything, we convert each $r_k$ as follows:

$$\tilde{r}_k \;=\; r_k \;+\; s\,\|r_k\|\,\frac{h_k}{\|h_k\|}, \qquad k = 1, \ldots, K.$$

Notes that the oracle's $r_k$ is a part of the sum, and the target contributes only the direction of $h_k$, rescaled to the length of $r_k$. Rescaling to $\|r_k\|$ keeps what the oracle reads inside the range its training put it in. Karvonen et al. (2025) add the target direction instead of overwriting $r_k$ with it because overwriting made the norm at these positions grow by about five orders of magnitude during training at the injection layer. The scalar $s$ is the *steering coefficient* and defaults to 1. The component that performs this addition during generation is the *steering hook*, and decoding *under the hook* means generating while the addition is in place.

Concretely, the oracle is an instruction-tuned chat model carrying a low-rank adapter (Hu et al., 2021, LoRA). The same chat model, carrying a different adapter, serves as the target model, so the oracle and the target share their weights apart from the two adapters. Karvonen et al. (2025) train the oracle adapter on roughly one million constructed examples, each pairing a recorded set of activations with a desired output. Those proxy tasks include question answering about system prompts, binary classification, and prediction of the tokens surrounding the recorded activations.

## 3.2 Secret-word model organisms

A key challenge for an auditor is deception, whereby a model deliberately makes wrong statements despite knowing better. A secret-word model organism reproduces that challenge under controlled conditions where we can ascertain that the model has the right answer, but is trained to not reveal it. Each target model is fine-tuned to drop hints about one word but to refuse requests to name it directly. This way the word is encoded in the target model's weights but stays out of the target's output by design. The auditor's question, whether the model knows something it is not saying, then has an exact answer – resembling a controlled experimental setup for studying deceptive behavior and calibration of activation oracles.

Specifically, we build upon the 20-word secret-word task of Karvonen et al. (2025). Twenty copies of the target model are created, each fine-tuned with a LoRA adapter to keep one particular word secret while otherwise chatting normally. Two prompts define one sample: the *context prompt*